

Quick Ans: Docker

◆ Basics

1. What is Docker, and how is it different from a virtual machine?

- **Docker** = Containerization platform that packages apps + dependencies.
- **Containers** share the **host OS kernel** → lightweight, fast startup.
- **Virtual Machines** = Full OS + hypervisor → heavier, slower.
- **Key Difference:** Containers virtualize the **OS**, VMs virtualize the **hardware**.

2. Explain Docker images vs. containers.

- **Image:** Immutable blueprint of an application (read-only).
- **Container:** Running instance of an image (read-write).
- Analogy: Image = class, Container = object instance.

3. What is Docker Hub, and how is it used?

- Public registry for storing and sharing Docker images.
- Commands:

```
docker pull <image>  
docker push <image>
```

-
- Used to distribute images across teams or deploy to production.

4. What is a Dockerfile, and what is its purpose?

- Text file containing **instructions to build a Docker image**.
 - Example: FROM openjdk:21-jdk-slim, COPY, RUN, CMD.
 - Purpose: Automate image creation consistently.
-

5. Explain the purpose of docker build, docker run, and docker ps.

- `docker build -t <name> .` → Creates an image from Dockerfile.
- `docker run <image>` → Runs a container from the image.
- `docker ps` → Lists running containers.

6. How do you remove all stopped containers and unused images?

```
docker container prune    # Remove stopped containers
docker image prune -a     # Remove unused images
# Or combine:
docker system prune -a
```

7. What is the default network driver in Docker?

- **bridge** → Connects containers on a single host with NAT.

8. Explain the difference between CMD and ENTRYPOINT in Docker.

- **CMD**: Default arguments for container, can be overridden.
- **ENTRYPOINT**: Main executable, less easily overridden.
- **Best practice**: Use ENTRYPOINT for app, CMD for default args.

9. What are volumes in Docker, and why are they important?

- Volumes = **persistent storage** for containers.
- Data persists **even if container is deleted**.
- Mount using `-v <volume_name>:/path`.

10. How do you inspect logs of a running container?

```
docker logs <container_id_or_name>
# Stream logs live
docker logs -f <container_id>
```

◆ Medium Level (Hands-On & Practical)

1. What are Docker namespaces and cgroups?

- **Namespaces** → Isolate containers (PID, network, filesystem).
 - **cgroups** → Control and limit container resources (CPU, memory).
-

2. How do you optimize the size of a Docker image?

- Use **slim/base images** (e.g., alpine).
 - **Multi-stage builds** to remove build-time dependencies.
 - Minimize layers, combine RUN commands.
 - Clean caches (apt-get clean).
-

3. Explain multi-stage build in Docker.

```
# Stage 1: build
FROM maven:3.9-jdk-21 AS builder
COPY . /app
RUN mvn clean package

# Stage 2: runtime
FROM openjdk:21-jdk-slim
COPY --from=builder /app/target/app.jar app.jar
CMD ["java","-jar","app.jar"]
```

- Reduces final image size, separates build/runtime.
-

4. How would you handle environment variables in Docker?

- **Dockerfile:** ENV VAR_NAME=value
 - **docker run:** -e VAR_NAME=value
 - **Docker Compose:** under environment: section.
-

5. What are Docker Compose and its benefits?

- Tool to define **multi-container applications** in docker-compose.yml.
 - Benefits:
 - One command to spin up multiple containers.
 - Define networks, volumes, dependencies.
 - Simplifies dev/test environments.
-

6. How do you configure networking between multiple containers?

- Use **Docker networks**: docker network create <name>
 - Attach containers to same network → can communicate by **container name**.
 - Docker Compose automatically creates a network per project.
-

7. Explain bind mounts vs. volumes.

- **Bind mounts**: Host directory directly mapped → simple, but OS-dependent.
 - **Volumes**: Managed by Docker → portable, works on all platforms, safer.
-

8. How to persist data in Docker containers?

- Use **volumes** or **bind mounts**.
- Example:

```
docker run -v mydata:/var/lib/mysql mysql
```

- Data persists even if container is removed.
-

9. How do you troubleshoot a failing Docker container?

- Check logs: docker logs <container>
 - Check status: docker ps -a
 - Inspect container: docker inspect <container>
 - Test with interactive shell: docker run -it <image> /bin/bash
-

10. Difference between docker-compose up and docker-compose up --build

- `up` → Starts existing images.
 - `up --build` → Rebuilds images before starting.
-

◆ **Advanced Level (Production & Scaling)**

1. Explain the architecture of Docker (dockerd, containerd, runc).

- **dockerd** → Docker daemon, manages containers, images, networks.
 - **containerd** → Container runtime, manages container lifecycle.
 - **runc** → Low-level runtime, creates containers using OS primitives.
-

2. How do you secure Docker containers in production?

- Run containers as **non-root users**.
 - Limit capabilities (`--cap-drop`).
 - Use **read-only file systems**.
 - Sign images and scan for vulnerabilities.
 - Limit resource usage (cgroups).
-

3. What are Docker Swarm and Kubernetes? How do they compare?

- **Docker Swarm**: Native Docker clustering, simpler, less features.
 - **Kubernetes**: Full-featured orchestration, scaling, self-healing.
 - Swarm → easier setup, less production-grade; K8s → industry standard.
-

4. Explain Docker overlay networks and their use cases.

- Overlay networks allow **multi-host container communication**.
 - Used in **Swarm or multi-node Docker setups**.
 - Enables containers on different hosts to communicate via virtual network.
-

5. How do you use Docker in CI/CD pipelines?

- Build Docker images in CI → Push to registry → Deploy containers in CD.

- Tools: Jenkins, GitHub Actions, GitLab CI/CD, ArgoCD.
 - Example: Maven build → Docker image → Kubernetes deployment.
-

6. Best practices for writing a Dockerfile

- Use **lightweight base images**.
 - Minimize layers (combine RUN commands).
 - Use **.dockerignore**.
 - Set **explicit versions** for dependencies.
 - Avoid storing secrets in Dockerfile.
-

7. How do you manage secrets in Docker?

- Docker **Secrets** (Swarm mode): docker secret create → mount in container.
 - Avoid environment variables for production secrets.
 - Use external secret managers (Vault, AWS Secrets Manager).
-

8. What is a scratch image, and when would you use it?

- **Scratch** = empty base image → smallest possible.
 - Use for **minimal runtime images**, often for Go, C++ apps.
-

9. How do you debug networking issues in a Dockerized environment?

- Inspect networks: docker network ls, docker network inspect <name>
 - Test connectivity: docker exec <container> ping <other>
 - Check port mappings: docker port <container>
 - Verify firewall rules and host IPs.
-

10. How do you handle container orchestration at scale?

- Use **Kubernetes** (preferred) or **Docker Swarm**.
 - Features: scaling, self-healing, rolling updates, service discovery.
 - Monitor with Prometheus/Grafana, logging with EFK stack.
-

✅ All your Docker questions are now **answered in a structured, interview-ready way.**

I can also make a **single Notion-ready copy-paste version** with all answers formatted and ready to insert.

Do you want me to do that next?